

CSD2161 / CSD2160 - 2026 SPRING - ASSIGNMENT 5

Online Trading Platform

Updated Design Report for Option 3

This report summarizes the latest state of the Visual Studio 2022 C++ trading platform, including the server, console client, stress tool, browser companion UI, and the new C++ localhost web host used to serve the web demo reliably across multiple tabs.

COURSE**CSD2161 /
CSD2160****DUE DATE****29 March 2026****SOLUTION****TradingPlatformCpp.sln****PRIMARY
DELIVERABLE****C++ server,
client,
stress, and
web host****COMPANION UI****TradingPlatformCpp.WebDemo****UPDATED****28 March 2026**

The official implementation remains the C++ Visual Studio solution. The browser UI is an instructor-allowed companion process/program used to present the same trading workflow more clearly without replacing the core C++ submission.

Erika Ishii

SIT ID 2402198

DigiPen ID erika.ishii

Yimo Kong

SIT ID 2403446

DigiPen ID yimo.kong

Elvis Lim

SIT ID 2401088

DigiPen ID
elvisshengjie.lim

1

Project Design Introduction

Server-authoritative trading over LAN with a presentation-ready web companion

This project implements a LAN-based online trading platform for Assignment 5 Option 3. Users connect to a TCP server, log in with usernames, inspect market and account snapshots, submit limit orders, cancel open orders, and observe fills and trade history across three symbols: ACME, BETA, and GAMMA.

The latest version keeps state mutation on the server so balances, holdings, fills, and order status transitions remain deterministic even under concurrent activity. The solution now also includes a lightweight C++ web host that serves the browser demo on `http://localhost:8080`. This avoids the unreliable cross-tab behavior that can happen when opening static HTML directly through `file:///`.

Primary user goals

- Reconnect and recover the same account safely.
- View balances, holdings, books, orders, and recent trades.
- Place validated buy and sell limit orders across multiple symbols.
- Preserve state across disconnects and server restarts.

Core design choices

- Line-based TCP protocol for predictable ordered delivery.
- Mutex-protected server-owned trading engine.
- Price/time-priority matching plus market-maker support.
- Text-file persistence through `trading_state.txt`.

Presentation companion

- Standalone browser demo with login and dashboard views.
- Multi-tab support when served from the C++ localhost host.
- Per-tab session state and shared exchange state for cleaner demos.
- Dedicated web test checklist captured in the README.

2

System Features and Current Behavior

What the latest platform delivers

AREA	CURRENT BEHAVIOR	LATEST UPDATE REFLECTED IN THIS REPORT
Login and account state	Username-based login, duplicate-login protection, reconnect-safe recovery.	Manual client testing confirms restored balances, holdings, and order history after reconnect.
Trading and validation	Buy and sell limit orders validate symbol, quantity, price, and available cash or holdings.	Manual tests confirmed exact-price matches, partial fills, cancellation, and validation errors.
Market visibility	Separate books and trade history for ACME, BETA, and GAMMA.	Symbol-isolation tests confirmed activity in one book does not affect the others.
Persistence	<code>trading_state.txt</code> stores accounts, orders, and recent trades for restart recovery.	Server restart recovery was validated with restored GAMMA partial orders and restored trade history.
Stress testing	Concurrent load generator simulates many TCP clients and rapid commands.	A recent successful run observed 264 commands, 151 accepted orders, 579 trade notices, and 1182 snapshots with zero failures.
Web companion UI	Login page, dashboard, portfolio, order entry, book view, trades, and activity feed.	The browser demo now uses shared local storage plus per-tab session storage and warns when opened through <code>file://</code> .
Web hosting	New C++ localhost host serves the web files without Python.	<code>TradingPlatformCpp.WebHost</code> is now part of the Visual Studio solution and serves the demo on port 8080.

Server authoritative

Three symbols

Persistent recovery

Busy communication support

Web demo on localhost

3

Implementation Structure

Clear separation between networking, engine logic, persistence, and interfaces

Networking layer

- `TradingPlatformCpp.Server` accepts TCP client connections and serves each session on its own thread.
- `TradingPlatformCpp.Common` centralizes protocol helpers and Winsock setup.
- `TradingPlatformCpp.WebHost` is a simple HTTP static-file host dedicated to serving the browser demo over localhost.

Trading logic layer

- Orders are validated before they enter the book.
- Matching uses price/time priority with market-maker fallback.
- Balances, reserved resources, orders, and trades are updated atomically.

Persistence layer

- State is saved to `trading_state.txt` after meaningful changes.
- Reconnects restore the same account state for the same username.
- Server restarts reload accounts, open orders, and trade history.

Interface layer

- The console client remains the official submission interface for functional testing.
- The web demo is a separate presentation-only surface with a cleaner trading dashboard.
- The new localhost host lets the web demo show reliable multi-tab behavior without external tooling.

The latest web architecture intentionally separates shared exchange data from per-tab session data. Shared state stays in browser local storage, while the active user and selected symbol stay in session storage so Alice and Bob can use separate tabs without overwriting one another.

4

Testing and Validation Summary

Manual walkthroughs, recovery checks, web-demo validation, and busy communication

TEST AREA	COVERAGE	OBSERVED RESULT
Manual console workflow	Login, symbol switching, order entry, partial fill, cancellation, history, and account inspection.	Passed. Clean-start runs matched expected balances, fills, and reserved-resource updates.
Reconnect recovery	Quit and reconnect the same user, then inspect balances, holdings, orders, and history.	Passed. Restored account state matched the saved server state.
Server restart recovery	Create GAMMA activity, restart the server, reconnect Alice and Bob, and inspect restored state.	Passed. Cash, holdings, partial orders, and trade history were restored from trading_state.txt.
Web demo workflow	Login, two-tab exact-price match, partial fill, cancellation, validation errors, symbol isolation, refresh persistence, and reset.	Covered in the updated README checklist and aligned with the current web implementation.
Stress test	8 clients, 30 operations per client, localhost server already running.	Passed with zero failures and zero errors.

264

commands sent

151

accepted orders observed

579

trade notices observed

1182

snapshots observed

One failed stress run earlier showed eight connection failures and zero commands, which confirmed the importance of starting the server before the stress tool. The later successful run validated the intended busy-communication behavior under load.

5

Technical Design Summary

Key implementation decisions and why they were kept in the latest revision

AREA	DESIGN CHOICE	REASON
Protocol	Line-based TCP messages	Parsing remains simple while delivery stays ordered and reliable.
Authority model	Server-owned state transitions	Clients cannot create conflicting balances, duplicate fills, or invalid order state.
Concurrency	Mutex-protected engine	Account, order, and trade updates stay atomic during concurrent requests.
Persistence	Text-file save/load workflow	Reconnect and restart recovery are supported without a database layer.
Web demo state model	Shared local storage plus per-tab session storage	Separate tabs can represent different users while still seeing the same exchange state.
Web serving	C++ static-file localhost host	Reliable same-origin browser behavior without introducing Python into the workflow.

Representative commands

- LOGIN|alice
- BUY|BETA|5|75.00
- SELL|GAMMA|2|125.00
- CANCEL|ORD0001
- REFRESH|ACME
- SYMBOL|BETA

Web companion operating notes

- Serve the browser demo from `http://localhost:8080` through `TradingPlatformCpp.WebHost`.
- Do not rely on `file://` when demonstrating multi-tab matching.
- The README now includes a dedicated web-demo test checklist with expected outcomes.

6

Individual Contribution and Conclusion

Shared ownership with updated web-demo and documentation scope

FULL NAME	SIT ID	DIGIPEN ID	CONTRIBUTION SUMMARY	CONTRIBUTION %
Erika Ishii	2402198	erika.ishii	Networking, project integration, server/client coordination, and submission alignment.	33%
Yimo Kong	2403446	yimo.kong	Trading engine logic, validation rules, persistence handling, and recovery testing.	33%
Elvis Lim	2401088	elvisshengjie.lim	Console UI polish, stress tooling support, browser demo improvements, C++ web host support, and documentation updates.	33%

The latest project version now presents a more complete and easier-to-demonstrate trading workflow. The official C++ solution remains the center of the assignment, while the browser companion has been strengthened with localhost hosting, better multi-tab behavior, and clearer test guidance. Together, the server, client, stress tool, web host, and web demo provide a coherent end-to-end system that demonstrates networking, synchronization, persistence, recovery, and user-facing clarity.

Generated from report/report.html to replace the older PDF snapshot and reflect the current repository state as of 28 March 2026.